

The Intelligent Bistro — Conversation Export

Project: Viridien AI Full-Stack Engineering Internship

Repository: `viridien_project_intelligent_bistro`

Export date: May 2026

Project links

- **Project design overview (video):** https://drive.google.com/file/d/1EBBY-bvrmypTlJ_AeMu5Q3dqiygNc_C/view
 - **Mobile experience walkthrough (Loom):** <https://www.loom.com/share/2b15ba7550ca4d30a38e7081d40c0484>
-

Part 1 — App features (one slide)

The Intelligent Bistro — AI-powered mobile ordering · Expo + Node.js

Menu & ordering

- 30+ menu items across 7 categories (Starters, Mains, Bowls, Salads, Sides, Drinks, Desserts)
- Size options on every dish (Small / Medium / Large) with dynamic pricing
- Category filters, descriptions, tags, manual add-to-cart from menu cards

Cart & checkout

- Live cart: quantity +/-, change size, remove lines, subtotal + 8% tax
- Place order from Cart tab or via AI with yes/no confirmation

Orders

- Orders tab: history, line items, totals, status
- Cancel from UI or chat ("Cancel my last order")

AI maître d' (chat)

- Natural-language ordering: add, remove, update quantity, change size
- Compound orders ("4 sandwiches, 7 burgers, with 3 lemonades")
- Menu Q&A with scrollable dish cards and one-tap Add
- Multi-category browse ("starters and bowls")
- Meal pairing and missing-category hints after adds
- Bulk quantity guard (> 10 per item → confirm before adding)
- Greeting and quick suggestion chips on the AI tab

Voice

- Web (Chrome/Edge): live speech-to-text on localhost
- Expo Go (iPhone/Android): record → Whisper via POST /api/transcribe

Platform & quality

- Cross-platform: iOS/Android (Expo Go) + web from one codebase
 - Rules + OpenAI: structured cart JSON, reconcile layer, offline-capable rules fallback
 - Dockerized API, health checks, 42 automated validation tests
-

Part 2 — 5-minute Loom presentation (3 slides)

Slide 1 — UI & Usability (~1:45)

On slide:

- Menu — 30+ dishes, 7 categories, size picker (S/M/L) with live pricing
- Cart — Edit quantity, change size, tax & total before checkout
- Orders — Placed orders, cancel from UI or chat
- Design — Dark bistro theme, gold accents, haptics on device
- Works everywhere — Expo Go (phone), web (localhost:8081), API on Docker/local

Demo: Menu → pick Large → Add → Cart → change size → flash Orders tab

Speaker notes: Guests order manually or via AI maître d'. Expo + React Native, one codebase. Every dish has S/M/L with category-based pricing.

Slide 2 — AI-driven cart (~2:00)

On slide:

- Add/remove/resize — "Add two large waters", "Change my burger to small"
- Compound orders — "4 sandwiches, 7 burgers, with 3 lemonades"
- Menu Q&A — "What are your starters?" → scrollable cards + Add chips
- Smart flows — Place order → yes/no; bulk qty (> 10) needs confirm
- Pairing tips after adds
- Voice — Web live speech; Phone record → Whisper

Demo script:

1. "What are your starters?"
2. "Add four spicy chicken sandwiches and seven truffle mushroom burgers with three craft lavender lemonade"
3. Open Cart — verify quantities
4. "Change my burger to large" (if in cart)
5. "Place order" → "yes" → Orders tab
6. Optional: mic demo

Slide 3 — Code structure & AI stack (~1:15)

Monorepo: mobile/ (Expo) + backend/ (Express)

AI flow:

- Cart parsing (primary): OpenAI GPT-4o — `openaiCartActions.ts`
- Safety net: `reconcileAiCartActions()` + rules — `orderSegmentParser.ts`
- Menu browse: rules + `openaiMenuIntent.ts`
- Voice (phone): Whisper — `POST /api/transcribe`

Design: Structured JSON actions → `cartStore.applyActions()`

Show: `backend/src/services/`, `GET /health`, validate scripts

Closing: OpenAI parses intent → rules reconcile → structured JSON drives cart.

Part 3 — One slide: code structure & AI stack (detail)

Repository layout

```
backend/  - Express API: menu, chat, transcribe
mobile/   - Expo: Menu, AI, Cart, Orders (Zustand)
```

Request flow (AI path)

```
POST /api/chat → chatOrchestrator → resolveCartActions
├─ openaiCartActions.ts (GPT-4o, JSON, temp 0)
├─ reconcileAiCartActions()
└─ orderSegmentParser.ts (fallback)
→ CartAction[] → mobile cartStore.applyActions()
```

AI tools

Component	Technology	Role
Cart parser (primary)	GPT-4o	Multi-item, sizes, with-clauses
Reconcile layer	TypeScript rules	Stops re-adding full cart
Rules parser	orderSegmentParser.ts	Offline fallback
Menu classifier	openaiMenuIntent.ts	Multi-category browse
Structured flows	chatOrchestrator.ts	Place/cancel/confirm
Voice (mobile)	Whisper	transcribeService.ts

Cart action types

ADD, REMOVE, UPDATE_QUANTITY, SET_MODIFIER, CLEAR

Mobile stores

menuStore, cartStore, ordersStore

Part 4 — AI cart + voice assistant (one slide)

How it works

Guest types or speaks → POST /api/chat → reply + CartAction[] → applyActions()

Cart actions understood

Intent	Example
Add (with size)	"Add two large sparkling waters"

Compound	"4 sandwiches, 7 burgers, with 3 lemonades"
Remove	"Remove one small truffle fries"
Change size	"Change my burger to large"
Place order	"Place order" → yes/no
Menu + add	"What are starters?" + dish cards

Voice

Platform	How
Web	Live Web Speech on localhost:8081
Expo Go	Record → POST /api/transcribe (Whisper)

Smart behaviors

- Recommendation cards after menu questions
- Pairing suggestions after small adds
- Bulk guard (> 10 qty)
- Anti-hallucination: "Add lemonade" adds only lemonade

Part 5 — Future scope, deployment, scalability

Deployment next steps

1. Deploy Docker API (Fly.io, Railway, Render, AWS) — HTTPS, OPENAI_API_KEY
2. Restrict CORS to production origin
3. EAS build for mobile — production API URL
4. Smoke test: /health, /api/chat, /api/transcribe
5. CI: npm run build, validate:parser, validate:ai
6. Rate limits, logging, billing alerts

Scalability

Area	MVP	Scale path
API	Single stateless container	N replicas behind LB
State	Client cart/orders	Redis + accounts + server cart
Menu	In-memory TS	CMS/PostgreSQL + CDN
AI	Sync OpenAI per message	Queue workers, tiered models
Voice	Whisper per upload	Streaming STT

Future scope

Payments, kitchen display (WebSocket), accounts, rich modifiers, analytics, i18n, embedding search, offline rules bundle

Part 6 — One minute speaker notes (UI + AI chat)

On the phone: four tabs — Menu, AI, Cart, Orders.

Menu: category filters, size selector, price updates before Add.

Cart: quantity, change size, tax, Place order.

Orders: history and cancel.

AI tab: type or mic. Phone uses Whisper transcribe; web uses live speech.

"What are your starters?" → text + recommendation cards with Add.

Compound orders and size changes work via structured cart actions.

Place order in chat → summary → yes to checkout.

Dark bistro theme; compact chat so menus don't hide the conversation.

Part 7 — Development work summary (session arc)

Features delivered

- 1. **Sizes on all dishes** — menuModifiers.ts, SizeSelector, SET_MODIFIER, sizeParser
- 2. **OpenAI-first cart** — openaiCartActions.ts + reconcileAiCartActions()
- 3. **Compound orders** — expandWithClauses for "with 3 lemonades"
- 4. **iOS chat UI fix** — ScrollView flexGrow, RecommendationBlocks layout
- 5. **Cart hallucination fix** — OpenAI re-adding entire cart on single add
- 6. **Expo Go voice** — transcribe route + Whisper
- 7. **Validation** — 19 parser tests + 23 AI tests
- 8. **Documentation** — README, SESSION_NOTES_2026-05-18.md

Challenges & solutions

Challenge	Solution
iOS chat cards stretched full screen	flexGrow: 0 on bubbles; nested scroll; inner View layout
OpenAI re-added whole cart	reconcile + read-only cart context in prompt
"7 burgers with 3 lemonades" wrong qty	expandWithClauses + OpenAI examples
Rules before OpenAI	resolveCartActions — OpenAI first

Part 8 — Backend architecture overview

How a message flows

```
POST /api/chat
→ routes/chat.ts (Zod)
```

```
→ aiService.processChatMessage()  
  └─ Greeting → getGreetingReply()  
  └─ handleStructuredChat()  
  └─ OpenAI fallback OR rulesFallback()  
→ ChatResponse → mobile applyActions()
```

File roles (summary)

File	Role
index.ts	Express app, CORS, routes, /health
types/index.ts	MenuItem, CartAction, ChatRequest/Response
data/menu.ts	30+ items, aliases
data/menuModifiers.ts	Size pricing, computeUnitPrice
routes/menu.ts	GET menu, categories
routes/chat.ts	POST chat validation
routes/transcribe.ts	POST audio → text
aiService.ts	processChatMessage entry
chatOrchestrator.ts	Confirmations, orders, cart, menu
openaiCartActions.ts	GPT cart parser
openaiMenuIntent.ts	Category classifier
orderSegmentParser.ts	Rules parser + reconcile
sizeParser.ts	Size extract/SET_MODIFIER
messageNormalizer.ts	Intent flags, cleanup
menuInquiry.ts	Menu Q&A responses
menuBrowseResolver.ts	Rules + OpenAI menu merge
mealSuggestions.ts	Categories, pairings, blocks
orderParser.ts	Cancel, list, detail orders
transcribeService.ts	Whisper
ruleBasedParser.ts	Legacy (not main path)

Part 9 — Backend file-by-file (step by step)

Step 1 — index.ts

1. Load dotenv

2. Create Express + CORS + JSON 16mb
3. GET /health → ai + voice mode
4. Mount /api/menu, /api/chat, /api/transcribe
5. Listen 0.0.0.0:3001

Step 2 — types/index.ts

Defines MenuItem, CartAction (ADD/REMOVE/UPDATE_QUANTITY/CLEAR/SET_MODIFIER), OrderAction, ChatRequest (message, history, cart, orders, session), ChatResponse (reply, actions, recommendationBlocks, placeOrderFromCart, parsedBy).

Step 3 — menuModifiers.ts

normalizeSizeOptionId, sizeDeltasForCategory, buildSizeModifier, withStandardSizeModifiers, computeUnitPrice, defaultModifiersForItem (medium default), formatModifiersLabel.

Step 4 — menu.ts

MENU_ITEMS_RAW → map withStandardSizeModifiers → MENU_ITEMS. getMenuitemById, getMenuCatalogForPrompt for OpenAI.

Step 5 — routes/menu.ts

GET / → all items. GET /categories → unique categories.

Step 6 — routes/chat.ts

Zod ChatBodySchema validates request. Calls processChatMessage. 400 on Zod error, 500 on failure.

Step 7 — routes/transcribe.ts

503 if no API key. Validates audioBase64. Returns { text }.

Step 8 — transcribeService.ts

Decode base64, max 12MB, Whisper whisper-1, language en.

Step 9 — messageNormalizer.ts

normalizeCompoundMessage (voice fixes). extractMenuInquiryText, extractAddText. messageHasAddIntent, messageHasRemoveIntent, messageHasCartMutation, messageHasMenuInquiry.

Step 10 — sizeParser.ts

extractSizeFromText, stripSizeWords, parseModifierChangeActions (regex SET_MODIFIER), parseSizeInquiryReply, sizeLabelForAction.

Step 11 — orderSegmentParser.ts (pipeline)

1. expandWithClauses — "with 3 X" → "and add 3 X"
2. normalizeOrderMessage

3. splitOrderSegments — split on and/comma/plus
4. segmentIntent — remove vs add
5. extractQuantity — leading/trailing/word numbers
6. matchMenuItem — score names + aliases
7. extractModifiers — size + item-specific
8. parseAddActionsFromMessage loop
9. parseAllCartActionsFromMessage — modifier changes + adds
10. dedupeCartActions — sum duplicate keys
11. reconcileAiCartActions — filter OpenAI hallucinations

Step 12 — openaiCartActions.ts

Dedicated GPT-4o prompt + catalog. temperature 0, JSON actions. validateCartActions, applyDefaultModifiers, reconcileAiCartActions.

Step 13 — openaiMenuIntent.ts

Classify menu categories from ambiguous messages. Returns MenuCategory[].

Step 14 — mealSuggestions.ts

detectMenuCategories, buildCategoryMenuResponse, buildMultiCategoryMenuResponse, ITEM_PAIRINGS, postAddAdviceStructured, analyzeMealGaps, suggestion chips.

Step 15 — menuInquiry.ts

buildMenuInquiryResponse tries: size Q&A, meal completion, pairing, categories, full menu, recommendations, item price.

Step 16 — menuBrowseResolver.ts

Merge rules categories + OpenAI categories for multi-category browse.

Step 17 — orderParser.ts

parseOrderActions (cancel last/all/#), orderDetailReply, orderListReply, buildOrderReply. wantsOrderDetail excludes cancel phrasing.

Step 18 — chatOrchestrator.ts

handleConfirmation (place_order, bulk_add yes/no). resolveCartActions (OpenAI then rules). buildCartMutationResponse (summary, pairing, bulk split). handleStructuredChat order: confirmation → orders → place → compound → menu → cart → cart view. getGreetingReply.

Step 19 — aiService.ts

processChatMessage: greeting → structured → menu browse → OpenAI general JSON → rulesFallback. validateActions, guardHighQuantityActions.

Step 20 — ruleBasedParser.ts

Legacy monolithic parser — not main entry path.

Step 21 — Validation scripts

validate-order-parser.ts (19 cases), validate-ai.ts (23 cases).

Part 10 — End-to-end example

Message: "add four spicy chicken sandwiches and seven truffle mushroom burgers with three craft lavender lemonade"

1. chat.ts validates
 2. handleStructuredChat → handleCartAdd
 3. resolveCartActions → OpenAI + reconcile (or rules after expandWithClauses)
 4. Three ADD actions: sandwich×4, burger×7, lemonade×3
 5. buildCartMutationResponse → "Done — added ..."
 6. Mobile applyActions → cart updated
-

Part 11 — Key design principles

1. **Structured output** — AI returns CartAction[], not only prose
 2. **Client-owned state** — cart and orders on phone; server stateless
 3. **OpenAI-first cart, rules-safe** — reconcile prevents cart re-add bugs
 4. **Rules-first lifecycle** — place order, cancel, yes/no confirmations
 5. **Validation in CI** — 42 automated tests
-

End of conversation export.